Project II: Memory

Albin Gullberg
Chalmers University of Technology
Göteborg, Sweden
albingul

***Abstract*—This project aims to develop a computer vision model capable of accurately recognizing cards used in a "Memory" game. The primary goal is to detect and classify individual cards in an image and determine whether two revealed cards form a matching pair. The system relies on SIFT (Scale-Invariant Feature Transform), for extracting and matching descriptors. Perspective correction and preprocessing techniques are applied to improve recognition under varying image conditions. Contour detection is used to locate and isolate individual cards from the input image before further processing. The model is trained on images of individual cards and achieves high accuracy in identifying cards and their matches.**

I. Method

When detecting and classifying all the cards in a given image of memory cards, a combination of binary threshold and SIFT was used.

A. Collection of data

The first step was collecting all the data. The cards were laid out on the floor of a computer room. Pictures were then taken of the cards with different cards turned up. The cards were structured in a 4x4 grid or laid out with arbitrary rotations, and the picture was either taken from above the cards, or with a camera pose change of between $0^o - 45^o$. Two of each individual card class was also cropped to be used as training data for SIFT.

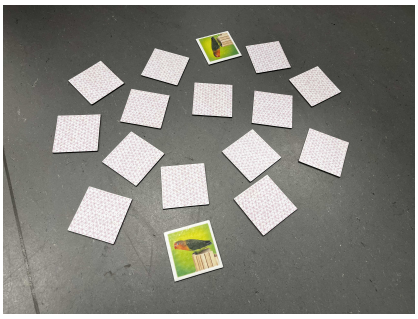


Fig. 1: Example of an image taken from an angle with rotated cards

B. Detecting the cards

To detect all the cards in the image, binary thresholding was used. Firstly, the color image is converted to grayscale. This is done because you want one binary threshold mask per image, and not one for each channel, which would be the case for the color image. After this, CLAHE, a type of local contrast enhancement, is used. This is done to minimize the negative effect of varying lighting, including shadows and glare. However, this will be further explained in section II. Following this, a gaussian blur is applied to minimize noise when then generating the threshold mask. In this step, Otsu's threshold is used to automatically find an appropriate threshold for the binary image.

The model then generates contours around the edges of the cards, where the white and black pixels of the binary image meet. These contours are then used to draw bounding boxes around the cards. To ensure that the contours are valid cards, a minimal area and four corner requirement must be met.

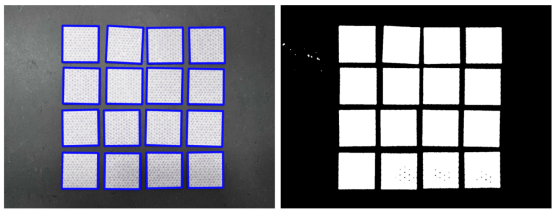


Fig. 2: Detected cards with bounding boxes and binary threshold image

C. Cropping and warping the cards

To prepare for the SIFT classifier, all the cards are individually cropped based on the contours. However, the cards are first warped to fit a square of 200x200 pixels. This is done to make the model more robust on the images with camera pose changes. This is done by first detecting the four corners of the contour around a card. Then each corner point is warped to the corresponding destination point. This warped card is then saved to a list which SIFT later uses to classify the card.



Fig. 3: Cropped card with and without warping

D. SIFT classifier

SIFT is used to classify each card. The SIFT functions were taken from the church classification in Lab 1. The classes are ”back”, for the face down card, and one class for each of the eight animals. The first step is to create training data. SIFT does this by extracting descriptors for each input card, with a label indicating which class the descriptor belongs to. It then extracts the same SIFT features from the card that is to be classified and matches its descriptors with those in the training data. After that, the classifier selects the class with the most matched descriptors by counting how many times each label appears among the matches.

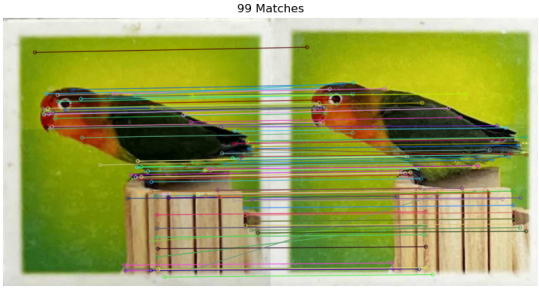


Fig. 4: SIFT matches between cropped card (left) and training image (right)

E. Use of LLMs

When building this model, LLMs were used as a tool, mainly to help find, understand, and implement OpenCV functions. They were also used to some extent for implementing other parts of the code, primarily to understand and learn how to write the code rather than to copy and paste it. However, the core ideas and the solutions to the problems along the way are mostly my own.

II. Experimental evaluation

A. Without warping when cropping cards

In this test, the model performed much worse on the images with a different camera perspective, as seen in I. An example of this is 5, where the SIFT classifier struggles to classify the cards correctly. During testing, it was also observed that each classification produced very few feature matches. This is because SIFT is not very good at change in camera perspective, especially when the change is big. After this was implemented in the code, the model performed perfectly on the images with a different perspective.

TABLE I: Quantitative Evaluation of Card Classification

Type of image	Accuracy (%)	Errors	Total cards
Normal	100	0	336
Rotated	100	0	48
Perspective	97.6	5	208
Persp. & Rot.	100	0	48
Total	99.2	5	640

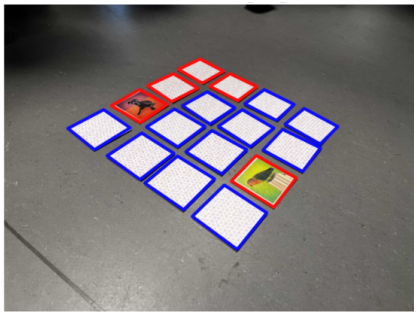


Fig. 5: Failure case without local contrast enhancement or quadrangle requirement

B. Without local contrast enhancement or quadrangle requirement

In this test, CLAHE, the local contrast enhancement, and the quadrangle requirement for a contour, were removed. this made the model much more sensitive to varying lighting, especially glare and shadows, which can be seen in the failure case in figure 6. It also made it possible for invalid shapes like triangles or pentagons to get through. After these features were implemented in the code, the model performed a lot better on images with varying lighting.

TABLE II: Quantitative Evaluation of Card Classification

Type of image	Accuracy (%)	Errors	Total cards
Normal	99.7	1	336
Rotated	100	0	48
Perspective	97.6	5	208
Persp. & Rot.	93.8	3	48
Total	98.6	9	640

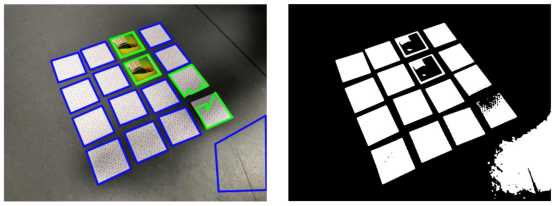


Fig. 6: Failure case without local contrast enhancement or quadrangle requirement

C. Without local contrast enhancement, quadrangle requirement, or gaussian blur

In this test, the model performed much worse. This is very expected because without gaussian blur, the noise in the threshold image becomes much more visible. This made it difficult for the model to detect the edges of the card accurately, which can be seen in figure 8. Gaussian blur is therefore an important part of the model in order for it to perform well. An interesting thing though, is that the failures reduced in the perspective images from the previous test, which can be seen in table III. This could be because the noise increased, which could have divided larger noise contours

into many smaller contours. Because there is a minimal area requirement for all contour boxes, these smaller areas could have been filtered out.

TABLE III: Quantitative Evaluation of Card Classification

Type of image	Accuracy (%)	Errors	Total cards
Normal	94.9	17	336
Rotated	95.8	2	48
Perspective	97.6	5	208
Persp. & Rot.	97.9	1	48
Total	96.1	25	640

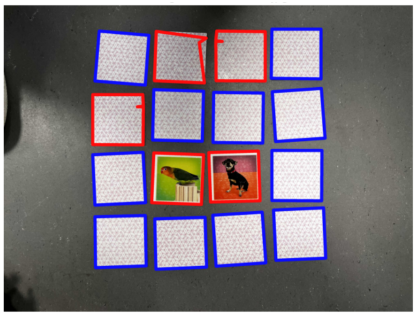


Fig. 7: Failure case without local contrast enhancement or quadrangle requirement

D. Final model

When each of these features was implemented in the code, the model performed very well, achieving an impressive accuracy of 100% on the tested dataset. However, it is possible that the model is still not entirely robust to variation in lighting, such as strong glare, heavy shadows, or backgrounds with light colors. These conditions may interfere with the binary threshold method for detecting the contours around the cards.

In conclusion, the model demonstrates strong performance under the tested conditions and shows promising potential. However, more testing would be required in order to determine how well the model would perform in more diverse, real life scenarios.

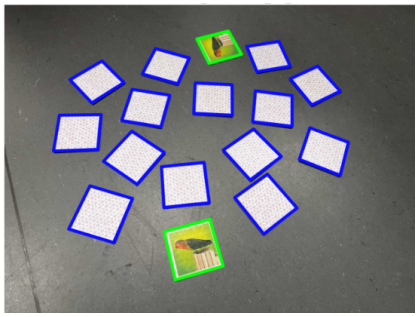


Fig. 8: Final model on image with perspective and rotated cards